

CV-SYNC

GitHub → AI Bullets → Compiled PDF
Presentation & Viva Preparation Guide

PART 1: PRESENTATION STRATEGY

1. The Hook (30 seconds)

"Your GitHub history is a better résumé writer than you are at 11pm the night before a deadline. CV-Sync watches repos you tag, extracts their README and metadata, uses AI to draft sharp bullet points, and never lets the LLM touch your actual LaTeX. You get a diff. You approve or reject it. Then it compiles to a PDF."

2. Problem Statement

Why CV-Sync Exists:

- Writing resume bullets is tedious, inconsistent, and error-prone
- You forget what you built 6 months ago — GitHub doesn't
- Existing tools either hallucinate LaTeX or trap you in a GUI
- No intermediate approval step = scary automated output

3. Key Differentiators

What	Why It Matters
LLM writes JSON, never LaTeX	Prevents hallucinated LaTeX commands. Zero PDF corruption risk.
Mandatory diff gate	You see exactly what changed before it lands in your CV.
RAG (Retrieval-Augmented)	Uses exemplar bullets + job description, not just your repo README.
8 compile-tested templates	Not screenshots—real LaTeX that works. Page-count previews built-in.

4. The Flow (Walkthrough)

Explain this as a sequence of user actions:

- 1. Sign in via GitHub OAuth (or email/password backup)
- 2. Pick repos from your GitHub account tagged for your CV
- 3. Choose a template from 8 real, compile-tested LaTeX designs
- 4. System fetches README + metadata from those repos
- 5. AI (via RAG) turns that into metric-aware bullets in JSON
- 6. System splices JSON bullets into the LaTeX template
- 7. You see a diff: "Here's what changed"

- 8. Approve → system compiles to PDF → download

5. Why Architecture Matters (What You're Doing Right)

- Four loosely-coupled services, not a monolith → easier to test, scale, hand off
- Auth service is separate → reusable, modular design
- Compiler service (by teammate) already done, proven
- Projects.json contract is the load-bearing wall → LLM and rendering talk through structured JSON only
- Session-based auth (not JWT) → revocation means something

PART 2: TECHNICAL DEEP DIVE (VIVA PREP)

A. Complete System Architecture

Four independent services communicate via contract (projects.json):

1. Auth Service (backend/auth-service)

- GitHub OAuth primary, email/password alternative, Google unmounted (future)
- Session-based: opaque session_id in cookie, everything server-side in Redis
- Routes: /api/auth/github, /email/login, /email/register, /me, /logout
- Status: Complete

2. Frontend (React + Vite)

- Onboarding flow: Auth → Experience → GitHub Repo Picker → Template Picker
- Template picker: 8-grid with multi-page preview modal, page-count badges
- Status: Complete

3. Fetch + LLM Engine

- Retrieval-Augmented Generation (RAG): pulls GitHub metadata, retrieves exemplar bullets, extracts job keywords
- Always outputs strict JSON (projects.json shape), never LaTeX
- Status: Designed, not yet built

4. Writer Service

- Takes projects.json → splices into LaTeX between % PROJECTS_START/END markers
- Calls Compiler Service to validate
- Returns diff to frontend (user approves/rejects)
- Status: Next up

5. Compiler Service (by Abhinav)

- POST /api/compile { tex } → PDF or 422 + line-numbered errors
- Sandboxed: latexmk in texlive Docker, paranoid file access (openin_any=p, openout_any=p), 15s timeout
- Status: Done

B. The Core Contract: projects.json

This is the load-bearing wall. LLM outputs this shape; Renderer reads this shape:

```
[{"title": "Legal Clause Conflict Resolver", "tech_stack": ["Python", "DeBERTa-v3", "FAISS", "FastAPI"], "bullets": ["Fine-tuned DeBERTa-v3-large on an NLI dataset, reaching 87%+ validation accuracy.", "Cut vector-search query latency 45% by optimizing the FAISS retrieval pipeline."], "repo_url": "...", "dates": "Jan 2026 – Present"}]
```

Why JSON?

- Structured, validated, zero ambiguity
- LLM can't slip a LaTeX command past the renderer
- Diff is readable to humans

C. Design Principles (The Rules)

These govern every decision in the codebase:

1. LLM never writes LaTeX. Ever.

It writes JSON. A deterministic renderer owns every \ in the output. Prevents hallucinations, injection, command escape breakouts.

2. There is always a diff gate.

No AI-authored change lands in your CV without human eyes seeing the exact diff first. Reject is default, safe, reversible.

3. Compile errors are a renderer bug, not an LLM problem.

A 422 from the compiler goes back to the code (developer debugging), never back into an LLM prompt. Prevents error loops.

4. Session auth, not JWT.

Revocation must mean something. Logout destroys the Redis session immediately, not just invalidates a cookie.

5. Static, pre-rendered previews over live rendering.

Template previews compiled once, committed as PNGs. No LaTeX toolchain needed at runtime just to show a picker grid.

D. Authentication (Session-Based Design)

Why Session, Not JWT?

- Session logout = immediate revocation (delete from Redis). JWT token still valid until expiry.
- Cookie is opaque; session details server-side → harder to forge
- Stateless APIs are hip, but safety-critical revocation is not stateless.

Flow:

- Sign up / login → verify (GitHub OAuth or argon2/bcrypt) → create Redis session (TTL) → Set-Cookie (last step, always) → 200
- Protected route → read cookie → session_id → Redis lookup → 200 or 401
- Logout → DELETE Redis session → clear cookie → 200

Security Details:

- Email/password uses bcrypt (12 rounds), no plaintext storage
- No user enumeration: login attempt on non-existent email runs dummy argon2 verify, returns same generic 401 as wrong password
- GitHub OAuth is primary, recommended; email is full alternative (not secondary)
- Google OAuth is implemented but intentionally unmounted (feature, not bug—finding it disconnected in code is expected)

E. Retrieval-Augmented Generation (RAG Design)

Why Retrieval, not Fine-Tuning?

Fast iteration: exemplars, job descriptions, and rules change; no retraining needed. Query-time retrieval adapts instantly.

Corpus	What	Retrieval
A. Template macros	~40 lines, 4 LaTeX constructs renderer understands	Always prompt-stuffed
B. ATS rules	Keyword hygiene, no tables/images, correct section names	Always prompt-stuffed
C. Bullet exemplars	100+ tagged	Embedded (text-embedding-3-

	(ML/backend/frontend/devops), growing	small), top-5 retrieved per repo
D. Target job description	User-pasted, optional	Chunked, tech-overlap retrieval, keyword extraction

[A + B] + [retrieved C exemplars + D keywords] + [repo data + your notes] → strict JSON out.

Implementation Status:

- No vector DB yet. JSON file of precomputed embeddings + cosine similarity is enough until corpus C grows into thousands.
- Exemplars and ATS rules already collected, waiting for service to be built.

F. Compiler Service (Sandboxing)

How Sandboxing Works:

- Docker image: texlive + latexmk, job directory created fresh per compile
- Paranoid file access: openin_any=p openout_any=p (block ../ traversal, read from /etc, etc.)
- No shell escape: -no-shell-escape flag on latexmk
- Hard 15s timeout → job killed, directory nuked
- Compiler errors max 10 reported, line-numbered, returned in 422 response body

API Contract:

- POST /api/compile { tex }
- 200 + binary PDF → success
- 422 + { ok: false, log, errors: [{file, line, message}] } → LaTeX error
- 400 → missing tex field
- 413 → tex > 2MB

G. CV Templates (8 Real, Tested)

- jake, dphang, anubhav: pdflatex + standard fonts
- altacv, moderncv: pdflatex + CTAN lato/roboto fonts
- plushcv, deedy, awesome-cv: xelatex + bundled fonts

Why Pre-Rendered Previews?

- Each template compiled once, all pages rasterized to PNG @150dpi
- Manifest.json maps template ID → page count
- Frontend template picker shows images (fast), page count badges, multi-page preview modal
- No LaTeX toolchain needed at runtime — it's all PNGs and metadata

H. Deployment & Infrastructure

Current Setup:

- Auth service: Node + Express, Redis for sessions, MySQL for user store
- Frontend: Vite + React
- Compiler: Docker image (texlive + latexmk), isolated via container
- Development: .env file (GitHub OAuth App credentials), local Redis + MySQL

Future Deployments:

- Docker Compose for local dev (Redis + MySQL + auth service containers)
- Kubernetes (or similar) for production (scale compiler service independently)
- CDN for template PNG previews + compiled PDFs

I. Key Design Decisions & Trade-Offs

1. Four services, not a monolith

Trade-off: More operational complexity, but each half testable independently. Critical when one half is being built by a teammate.

2. JSON contract, not direct LLM → LaTeX

Trade-off: Extra JSON validation layer, but zero hallucination risk and human-readable diffs. Worth it.

3. RAG (retrieval), not fine-tuning

Trade-off: Slower initial development, but exemplars and rules change constantly. Fine-tuning would require retraining.

4. Pre-rendered template previews, not live LaTeX rendering

Trade-off: Static previews can lag if template LaTeX changes. But runtime speed and simplicity win.

5. Session auth, not JWT

Trade-off: Requires Redis, slightly more complex. But logout means immediate revocation.

6. Sandboxed compiler in Docker, not Overleaf API

Trade-off: Own ops burden, but full control over sandbox, error reporting, and pricing.

J. Open Questions (What You're Still Figuring Out)

- Images vs. ATS-safe text-only CVs — genuinely unresolved, might end up as two template tracks
- Writer service: Node (assumed) or Python? Same repo, or its own?
- What happens the instant after a user clicks a template in the picker — no backend call exists yet; it's local React state until the writer service exists to receive it.

K. Security & Edge Cases

Attack Surface & Mitigations:

Attack	Mitigation
LLM injects LaTeX command	LLM only writes JSON, never touches .tex
Compiler escape (shell, file traversal)	Docker sandbox, paranoid file access perms, -no-shell-escape
Session hijacking	Opaque session_id, server-side Redis, HTTPS (in prod)
User enumeration (auth)	Dummy bcrypt verify on non-existent email, same generic 401 as wrong password
DoS (compiler resource exhaustion)	Hard 15s timeout, job directory auto-cleanup, 2MB tex limit

L. Demo Walkthrough (Live Presentation Script)

If you're demoing live:

- Step 1: Show login screen, explain session-based auth + GitHub OAuth
- Step 2: Pick repos → show how it reads your GitHub
- Step 3: Template picker → show 8-grid, multi-page preview, page-count badges
- Step 4: Explain what happens next (Fetch + LLM, not yet built, but RAG design complete)
- Step 5: Show projects.json shape → explain why JSON-only
- Step 6: Explain diff gate, approval flow, compiler sandbox
- Step 7: Show compiled PDF output, explain LaTeX toolchain was never exposed to LLM

M. Probable Viva Questions & Answers

Q1: Why is the LLM not allowed to write LaTeX?

A: LLMs hallucinate. They might invent a command (`\myCustomMacro`) that doesn't exist, or escape a string improperly, or nest braces incorrectly. These errors corrupt the PDF or cause a 422 compile error. By restricting the LLM to JSON, we guarantee it can't write invalid LaTeX—it can only produce data. The renderer (deterministic, testable code) owns all LaTeX.

Q2: Why session auth instead of JWT?

A: With JWT, a logout only invalidates the cookie; the token is still valid until expiry. With sessions, logout immediately deletes the Redis entry—revocation is instant and guaranteed. For a CV tool where you might revoke access to a teammate's account mid-project, that matters.

Q3: Why RAG and not fine-tuning?

A: Exemplars and ATS rules change all the time. Fine-tuning would require retraining + re-validation after every update. RAG retrieves at query time, so new exemplars or job descriptions take effect instantly. We start with a JSON file of embeddings; we scale to a vector DB if corpus C grows into thousands.

Q4: What if the compiler returns a 422?

A: The error (file, line, message) goes back to the developer or the rendering logic—never back into an LLM prompt for retry. We treat it as a bug in the renderer, not a reason to ask the LLM to "fix" things. This prevents infinite error loops and keeps the system deterministic.

Q5: How do you prevent template preview previews from going stale?

A: We regenerate them every time the template LaTeX changes. It's a one-time CLI script (render-template-previews.sh) that compiles all 8 templates, rasterizes to PNG, and writes a manifest.json. We commit the PNGs so no LaTeX toolchain is needed at runtime—just metadata.

Q6: How is the compiler sandboxed? What could go wrong?

A: Docker container with texlive + latexmk. Paranoid file access (openin_any=p, openout_any=p), no shell escape, hard 15s timeout, job directory auto-cleaned. The worst an attacker can do is consume CPU/memory or exhaust the 15s timeout. A 422 error is returned, job cleaned up, container ready for the next compile.

Q7: Why four services instead of a monolith?

A: Each service can be built, tested, and broken independently. Critical when teammates split: one builds auth/frontend, another builds the compiler, I build fetch+LLM and writer. The projects.json contract is the only coupling.

Q8: What is not yet built? What's the timeline?

A: Writer service (splice → diff gate → compile) and Fetch + LLM (RAG). Writer service is simpler (just string splicing and diff generation), then Fetch + LLM (GitHub API fetch, RAG retrieval, LLM call, JSON generation). The infrastructure (Redis, DB, auth, templates) is done; now it's building the data pipeline.

N. What Makes This Project Strong (Your Elevator Pitch for Interviewers)

- Safety-first: LLM is constrained (JSON only), never touches your PDF source
- Human in the loop: mandatory diff gate, user approval before compilation
- Modular architecture: four independent services, testable, scalable, team-friendly
- Real LaTeX, not screenshots: 8 templates compile-tested, multi-page, production-ready
- Thoughtful auth: session-based revocation, no user enumeration, bcrypt + GitHub OAuth
- Data-driven: RAG design scales with exemplars, job descriptions, no retraining
- Pragmatic sandboxing: Docker container, paranoid perms, reasonable timeouts

O. Summary: How to Talk About CV-Sync

Presentation (2-3 min):

"Tag repos on GitHub, CV-Sync pulls metadata, AI drafts bullets in JSON, you see a diff, you approve or reject, it compiles to PDF. The LLM never writes LaTeX—only data. A deterministic renderer handles all LaTeX. This keeps your PDF safe and your resume under your control."

Viva Deep Dive:

Walk through the four services (auth, compiler, templates, writer, fetch+LLM). Explain projects.json contract. Defend the design principles (LLM → JSON, diff gate, session auth, RAG). Describe the sandbox (Docker, paranoid perms, timeouts). Discuss trade-offs and open questions.